

In [1]:

```
# Importing vital packages -

import pandas as pd
from pandas.plotting import scatter_matrix
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import io
import locale
import statsmodels.api as sm
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
```

In [2]:

```
# Uploading data for google colab -

# from google.colab import files
# uploaded = files.upload()
# data = pd.read_csv(io.BytesIO(uploaded['shows_DB.csv']))
```

In [3]:

```
# Uploading data for vs code -
data = pd.read_csv('C:\\Users\\yarde\\Desktop\\[ פרוייקט בסטטיסטיקה ]\\shows_DB.csv')
data.head()
```

Out[3]:

	title	year	certificate	duration	genre	rating	description	stars	votes
0	Cobra Kai	(2018-)	TV-14	30 min	Action, Comedy, Drama	8.5	Decades after their 1984 All Valley Karate Tou...	['Ralph Macchio, ', 'William Zabka, ', 'Courtn...	177,031
1	The Crown	(2016-)	TV-MA	58 min	Biography, Drama, History	8.7	Follows the political rivalries and romance of...	['Claire Foy, ', 'Olivia Colman, ', 'Imelda St...	199,885
2	Better Call Saul	(2015- 2022)	TV-MA	46 min	Crime, Drama	8.9	The trials and tribulations of criminal lawyer...	['Bob Odenkirk, ', 'Rhea Seehorn, ', 'Jonathan...	501,384
3	Devil in Ohio	(2022)	TV-MA	356 min	Drama, Horror, Mystery	5.9	When a psychiatrist shelters a mysterious cult...	['Emily Deschanel, ', 'Sam Jaeger, ', 'Gerardo...	9,773
4	Cyberpunk: Edgerunners	(2022-)	TV-MA	24 min	Animation, Action, Adventure	8.6	A Street Kid trying to survive in a technology...	['Zach Aguilar, ', 'Kenichiro Ohashi, ', 'Emi ...	15,413

Preprocessing -

In [4]:

```
# Classify certificate into 4 groups -

data['TV-PG'] = np.zeros(len(data))
data['TV-TE'] = np.zeros(len(data))
data['TV-MA'] = np.zeros(len(data))
data['TV-NA'] = np.zeros(len(data))

for i, c in enumerate(data['certificate']):
```

```

if (c == 'PG-13') | (c == 'TV-PG') | (c == 'PG') | (c == 'TV-Y7-FV') | (c == 'TV-Y7')
) | (c == 'TV-Y') | (c == 'E10+') | (c == '12') | (c == 'TV-G') | (c == 'G') :
    data.loc[i, 'TV-PG'] = 1

if (c == 'TV-14') | (c == 'MA-17') | (c == 'NC-17') :
    data.loc[i, 'TV-TE'] = 1

if (c == 'TV-MA') | (c == 'M') | (c == 'R') | (c == 'Unrated') | (c == 'Not Rated')
:
    data.loc[i, 'TV-MA'] = 1

if (c == 'Approved') | (c == 'Passed') :
    data.loc[i, 'TV-NA'] = 1

```

In [5]:

```

# Dropping not useful attributes -

data_numer = data.drop(['certificate', 'description'], axis = 1)

```

In [6]:

```

# Turning the attribute 'votes' into int -

data_numer['votes'] = data_numer['votes'].str.replace(',', '').astype(float)

```

In [7]:

```

data_numer['stars'] = data_numer['stars'].apply(lambda x: len(x)).astype(int)

```

In [8]:

```

data_numer = data_numer.dropna()

```

In [9]:

```

# Turning the attribute 'duration' into int -

data_numer['duration'] = data_numer['duration'].str.replace(' min', '').astype(int)

```

In [10]:

```

# Turning the attribute 'year' into int(start year) -

roman_numerals = ['(I)', '(II)', '(III)', '(IV)', '(V)', '(VI)', '(VII)', '(VIII)', '(IX)',
                  '(X)', '(XI)', '(XII)', '(XIII)', '(XIV)', '(XV)', '(XVI)', '(XLI)', '(XXIII)', '(XXIV)', '(XXIX)']
problematic_words = ['(Video)', '(TV Movie)', '(TV Special)']

for num in roman_numerals:
    data_numer['year'] = data_numer['year'].apply(lambda x : x.replace((num + ' '), ''))

for num in roman_numerals:
    data_numer['year'] = data_numer['year'].apply(lambda x : (x.replace(num, '(1111)'))

for word in problematic_words:
    data_numer['year'] = data_numer['year'].apply(lambda x : (x.replace((word + ' '), '')))

for word in problematic_words:
    data_numer['year'] = data_numer['year'].apply(lambda x : (x.replace(word, '(1111)'))

data_numer['year'] = data_numer['year'].apply(lambda x : int(x.replace(x, x[1 : 5])))

```

In [11]:

```

# OHE the genres -

genre_lst = ['Drama', 'Action', 'Adventure', 'Animation', 'Horror', 'History', 'Comedy',

```

```
'Crime', 'Biography', 'Romance', 'Fantasy', 'Thriller', 'Sci-Fi', 'Sport', 'Mystery', 'Short', 'Family', 'Documentary', 'War', 'Music', 'Reality-TV', 'Game-Show', 'Talk-Show', 'Western', 'Film-Noir', 'News', 'Musical']

for genre in genre_lst :
    data_numer[genre] = data_numer['genre'].str.contains(genre).astype(int)

data_numer = data_numer.drop('genre', axis = 1)
```

In [12]:

```
# Merging Geners -

data_numer['Documentary'] += data_numer['Biography']
data_numer = data_numer.drop('Biography', axis = 1 )

data_numer['Action'] += data_numer['Thriller']
data_numer['Action'] += data_numer['Western']
data_numer['Action'] += data_numer['Film-Noir']

data_numer['Sci-Fi'] += data_numer['Fantasy']
data_numer['Drama'] += data_numer['War']
data_numer['Drama'] += data_numer['Mystery']

data_numer['Reality-TV'] += data_numer['Talk-Show']
data_numer['Reality-TV'] += data_numer['News']
data_numer['Reality-TV'] += data_numer['Game-Show']
data_numer['Reality-TV'] += data_numer['Music']

data_numer = data_numer.drop(['Thriller', 'Western', 'Film-Noir', 'Fantasy', 'War', 'Mystery', 'Music', 'News', 'Talk-Show', 'Game-Show', 'Short'], axis = 1 )
```

In [13]:

```
# Correcting the OHE -

genres = ['Drama', 'Action', 'Adventure', 'Animation', 'Horror', 'History', 'Comedy', 'Crime', 'Romance', 'Sci-Fi', 'Sport', 'Family', 'Documentary', 'Reality-TV', 'Musical']
data_numer[genres] = data_numer[genres].apply(lambda x : x.apply(lambda y : 0 if y == 0 else 1))
```

In [14]:

```
# Creating the Popularity feature -

C, M = 7, 25000
data_numer['popularity'] = ((data_numer['rating'] * data_numer['votes']) + (C * M)) / (data_numer['votes'] + M)
data_numer['popularity'].min()
print(f"The minimum value of popularity is {data_numer['popularity'].min()}")

The minimum value of popularity is 4.110898475510866
```

In [15]:

```
data_numer.drop(['rating', 'votes'], axis = 1)
```

Out[15]:

	title	year	duration	stars	TV-PG	TV-TE	TV-MA	TV-NA	Drama	Action	...	Comedy	Crime	Romance	Sci-Fi	Sport	Far
0	Cobra Kai	2018	30	80	0.0	1.0	0.0	0.0	1	1	...	1	0	0	0	0	
1	The Crown	2016	58	70	0.0	0.0	1.0	0.0	1	0	...	0	0	0	0	0	
2	Better Call Saul	2015	46	74	0.0	0.0	1.0	0.0	1	0	...	0	1	0	0	0	
3	Devil in Ohio	2022	356	78	0.0	0.0	1.0	0.0	1	0	...	0	0	0	0	0	
4	Cyberpunk: Edgerunners	2022	24	64	0.0	0.0	1.0	0.0	0	1	...	0	0	0	0	0	

...	title	year	duration	stars	TV: PG	TV: TE	TV: MA	TV: NA	Drama	Action	...	Comedy	Crime	Romance	Sci- Fi	Sport	Far
9952	The Imperfects	2022	45	81	0.0	0.0	1.0	0.0	1	1	...	0	0	0	0	0	
9953	The Walking Dead	2010	44	76	0.0	0.0	1.0	0.0	1	1	...	0	0	0	0	0	
9954	The Crown	2016	58	70	0.0	0.0	1.0	0.0	1	0	...	0	0	0	0	0	
9955	Supernatural	2005	44	73	0.0	1.0	0.0	0.0	1	0	...	0	0	0	1	0	
9956	Devil in Ohio	2022	356	78	0.0	0.0	1.0	0.0	1	0	...	0	0	0	0	0	

7673 rows x 24 columns

In [16]:

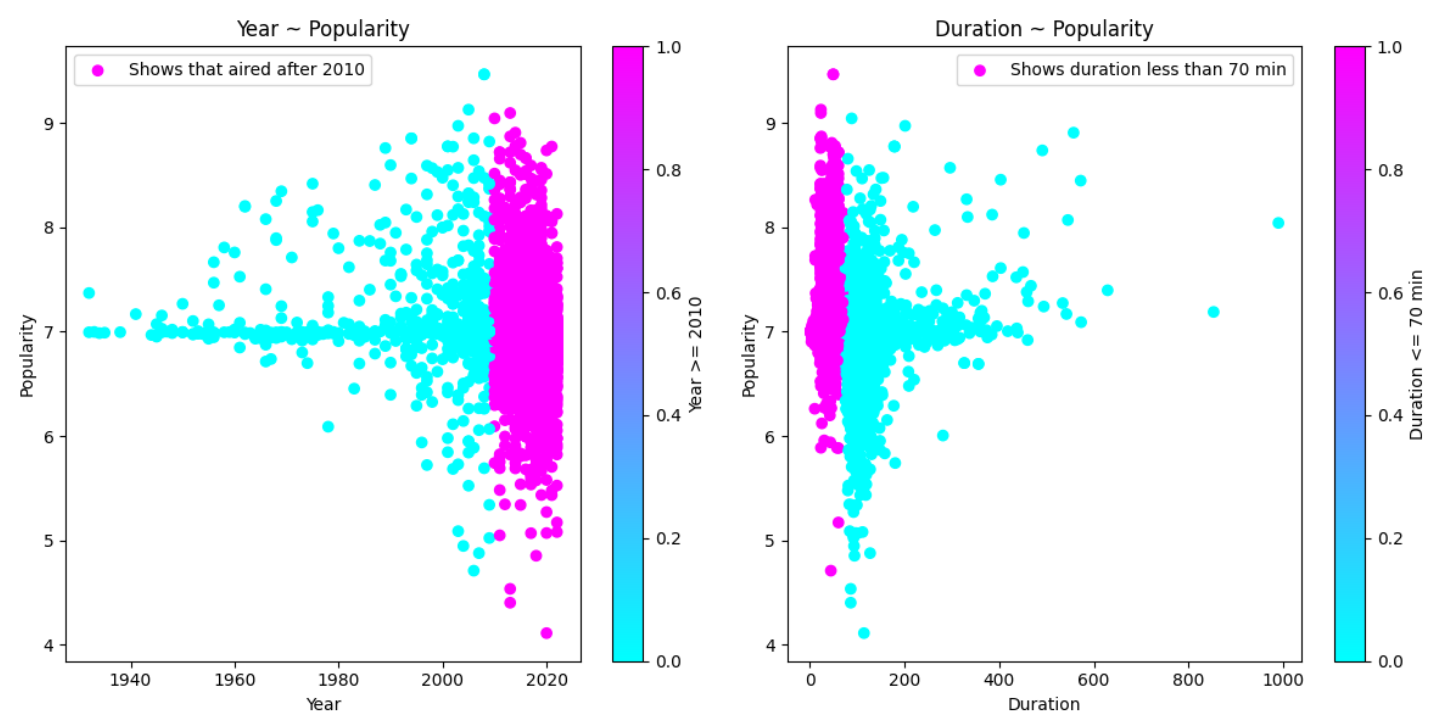
```
fig, axes = plt.subplots(1, 2, figsize=(12, 6))

scatter1 = axes[0].scatter(data_numer['year'], data_numer['popularity'], c=data_numer['year'] >= 2010, cmap='cool')
axes[0].legend(['Shows that aired after 2010'])
axes[0].set_title("Year ~ Popularity")
axes[0].set_xlabel('Year')
axes[0].set_ylabel('Popularity')

scatter2 = axes[1].scatter(data_numer['duration'], data_numer['popularity'], c=data_numer['duration'] <= 70, cmap='cool')
axes[1].legend(['Shows duration less than 70 min'])
axes[1].set_title("Duration ~ Popularity")
axes[1].set_xlabel('Duration')
axes[1].set_ylabel('Popularity')

cbar1 = plt.colorbar(scatter1, ax=axes[0])
cbar1.set_label('Year >= 2010')
cbar2 = plt.colorbar(scatter2, ax=axes[1])
cbar2.set_label('Duration <= 70 min')

plt.tight_layout()
plt.show()
```



In [17]:

```
# cutting off too old or too long contents -

data_numer['year'] = data_numer['year'][data_numer['year'] >= 2010]
data_numer['duration'] = data_numer['duration'][data_numer['duration'] <= 70]
```

In [18]:

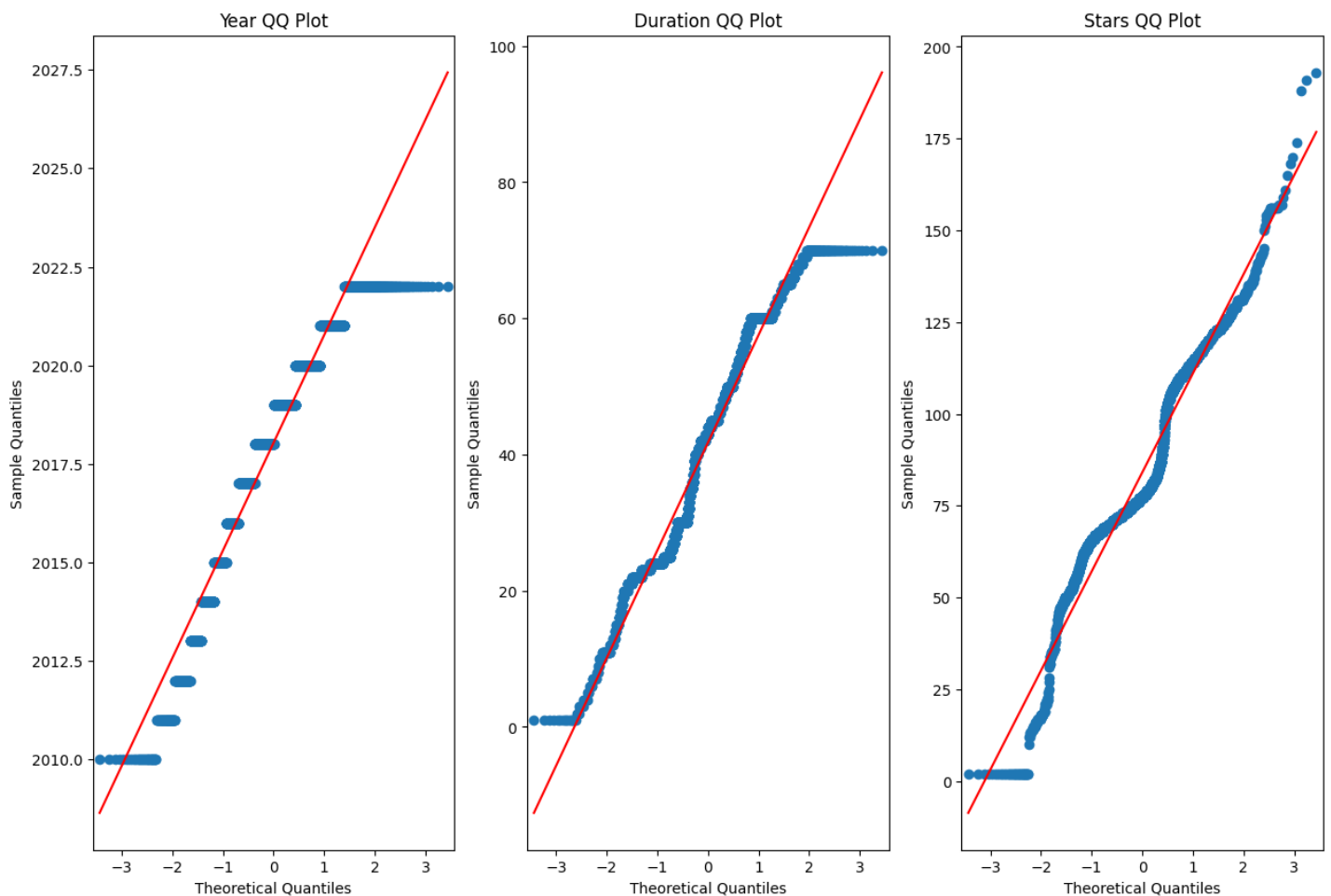
```
data_numer = data_numer.dropna()
```

The Research -

In [19]:

```
# Presenting QQ-Plots :

fig, ax = plt.subplots(nrows = 1, ncols = 3, figsize = (15 , 10))
axs = ax.flatten()
sm.qqplot(data_numer['year'], ax = axs[0], line='s')
ax[0].set_title('Year QQ Plot')
sm.qqplot(data_numer['duration'], ax = axs[1], line='s')
ax[1].set_title('Duration QQ Plot')
sm.qqplot(data_numer['stars'], ax = axs[2], line='s')
ax[2].set_title('Stars QQ Plot')
# sm.qqplot(data_numer['popularity'], ax = axs[3], line='s')
# ax[3].set_title('Popularity QQ Plot')
plt.show()
```

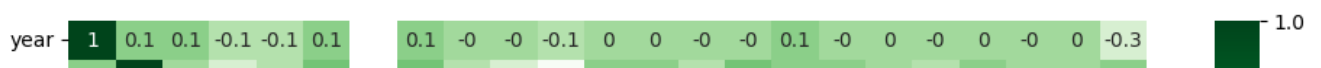


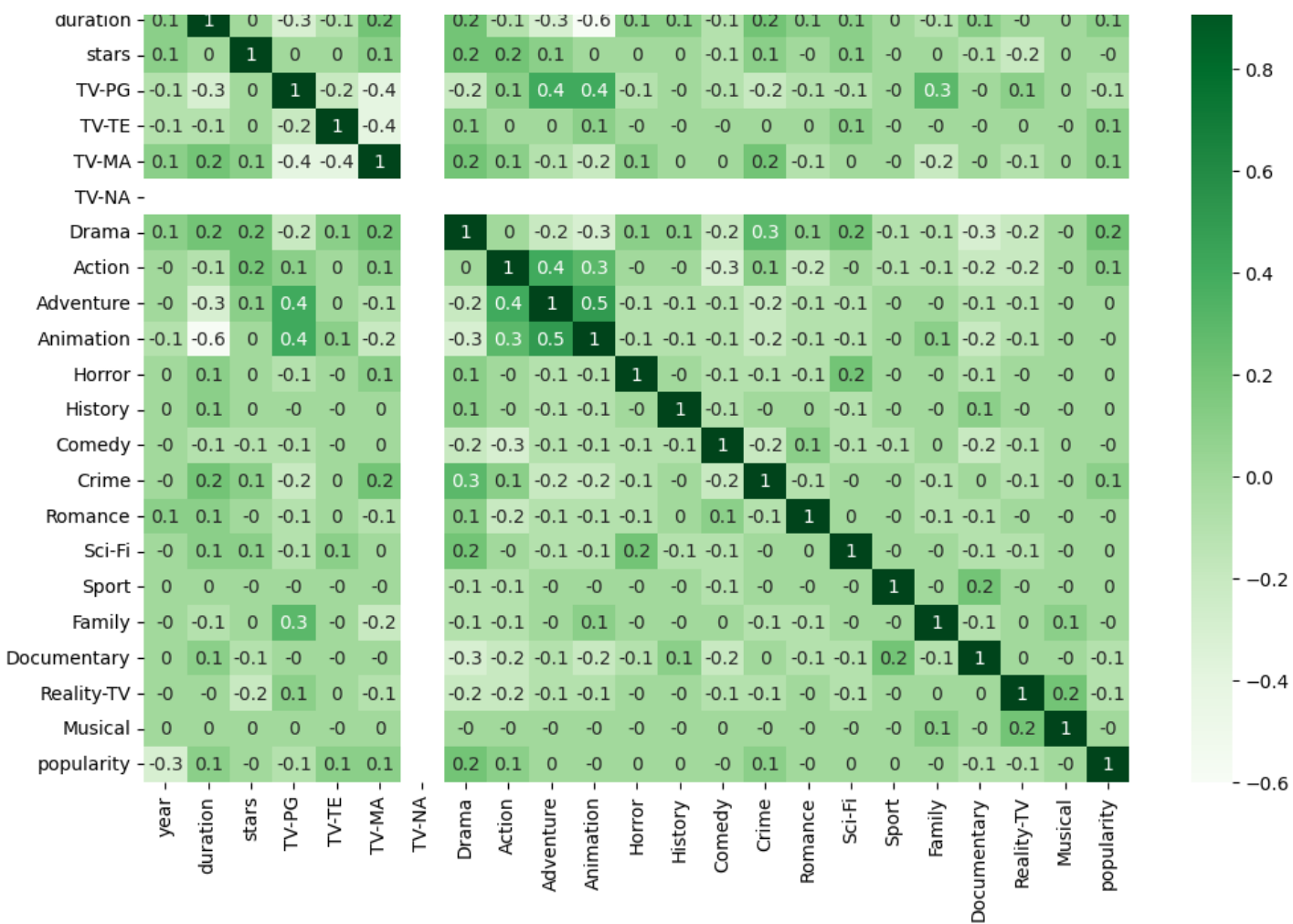
In [20]:

```
fig, ax = plt.subplots(figsize = (12, 8))
sns.heatmap(np.round(data_numer[['year', 'duration', 'stars', 'TV-PG', 'TV-TE', 'TV-MA', 'TV-NA', 'Drama', 'Action', 'Adventure', 'Animation', 'Horror', 'History', 'Comedy', 'Crime', 'Romance', 'Sci-Fi', 'Sport', 'Family', 'Documentary', 'Reality-TV', 'Musical', 'popularity']].corr(), 1), cmap = 'Greens', annot = True)
```

Out[20]:

<Axes: >





In [21]:

```
features = ['year', 'duration', 'stars', 'TV-NA', 'TV-PG', 'TV-TE', 'TV-MA', 'Drama', 'Action', 'Adventure', 'Animation', 'History', 'Horror', 'Comedy', 'Crime', 'Romance', 'Sci-Fi', 'Sport', 'Family', 'Documentary', 'Reality-TV', 'Musical']

model_features = data_numer[features]
model_target = data_numer['popularity']
```

In [22]:

```
data_numer.head()
```

Out[22]:

	title	year	duration	rating	stars	votes	TV-PG	TV-TE	TV-MA	TV-NA	...	Comedy	Crime	Romance	Sci-Fi	Sport	Fan
0	Cobra Kai	2018.0	30.0	8.5	80	177031.0	0.0	1.0	0.0	0.0	...	1	0	0	0	0	
1	The Crown	2016.0	58.0	8.7	70	199885.0	0.0	0.0	1.0	0.0	...	0	0	0	0	0	
2	Better Call Saul	2015.0	46.0	8.9	74	501384.0	0.0	0.0	1.0	0.0	...	0	1	0	0	0	
4	Cyberpunk: Edgerunners	2022.0	24.0	8.6	64	15413.0	0.0	0.0	1.0	0.0	...	0	0	0	0	0	
5	The Sandman	2022.0	45.0	7.8	80	116358.0	0.0	0.0	1.0	0.0	...	0	0	0	1	0	

5 rows x 26 columns

In [23]:

```
linear_features = sm.add_constant(model_features)
model = sm.OLS(model_target, linear_features)
```

```
model = model.fit()
print(model.summary())
```

OLS Regression Results

```
=====
Dep. Variable:          popularity      R-squared:          0.155
Model:                  OLS            Adj. R-squared:       0.150
Method:                 Least Squares   F-statistic:         29.47
Date:                   Wed, 21 Jun 2023 Prob (F-statistic):   3.73e-107
Time:                   15:17:15        Log-Likelihood:      -50.455
No. Observations:      3391           AIC:                144.9
Df Residuals:          3369           BIC:                279.7
Df Model:               21
Covariance Type:       nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
const	60.5817	3.197	18.950	0.000	54.314	66.850
year	-0.0266	0.002	-16.761	0.000	-0.030	-0.023
duration	0.0012	0.000	3.588	0.000	0.001	0.002
stars	-0.0012	0.000	-7.240	0.000	-0.002	-0.001
TV-NA	-9.282e-13	4.9e-14	-18.949	0.000	-1.02e-12	-8.32e-13
TV-PG	0.0345	0.016	2.225	0.026	0.004	0.065
TV-TE	0.1118	0.014	7.970	0.000	0.084	0.139
TV-MA	0.0987	0.012	8.421	0.000	0.076	0.122
Drama	0.0880	0.011	7.686	0.000	0.066	0.110
Action	0.0061	0.012	0.504	0.614	-0.018	0.030
Adventure	0.0474	0.015	3.151	0.002	0.018	0.077
Animation	0.0382	0.016	2.449	0.014	0.008	0.069
History	0.0332	0.024	1.370	0.171	-0.014	0.081
Horror	0.0222	0.024	0.914	0.361	-0.025	0.070
Comedy	0.0068	0.011	0.599	0.549	-0.015	0.029
Crime	0.0579	0.013	4.490	0.000	0.033	0.083
Romance	0.0122	0.016	0.773	0.440	-0.019	0.043
Sci-Fi	-0.0175	0.015	-1.185	0.236	-0.046	0.011
Sport	0.0776	0.036	2.177	0.030	0.008	0.147
Family	0.0066	0.022	0.305	0.761	-0.036	0.049
Documentary	0.0053	0.016	0.335	0.738	-0.026	0.036
Reality-TV	-0.0459	0.019	-2.382	0.017	-0.084	-0.008
Musical	0.0235	0.064	0.366	0.715	-0.103	0.150

```
=====
Omnibus:                1745.019      Durbin-Watson:         1.416
Prob(Omnibus):          0.000         Jarque-Bera (JB):      22743.831
Skew:                   2.136         Prob(JB):              0.00
Kurtosis:               14.947        Cond. No.              4.53e+20
=====
```

Notes:

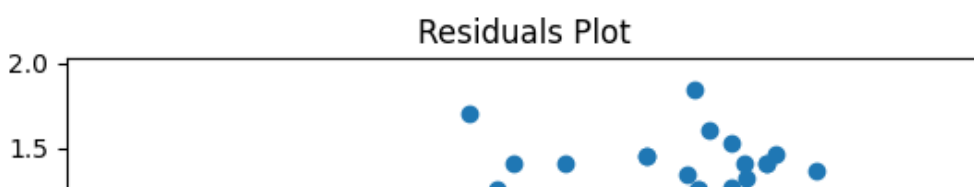
- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The smallest eigenvalue is 6.75e-32. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

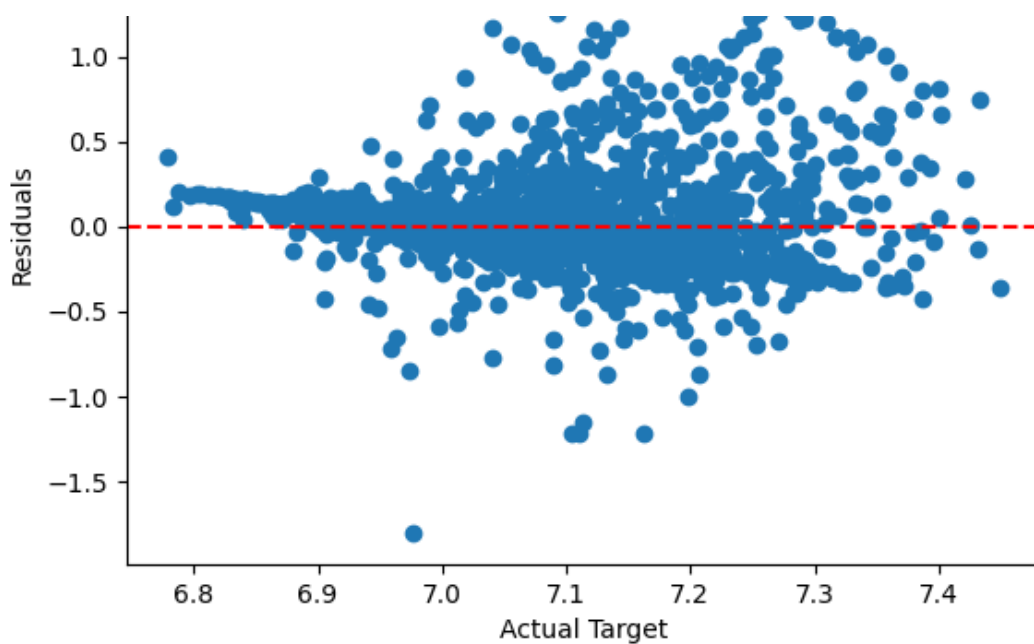
In [24]:

```
# Residuals plot -

predictions = model.predict(linear_features)

plt.scatter(predictions, model.target - predictions)
plt.axhline(0, color = 'red', linestyle = '--')
plt.xlabel('Actual Target')
plt.ylabel('Residuals')
plt.title('Residuals Plot')
plt.show()
```





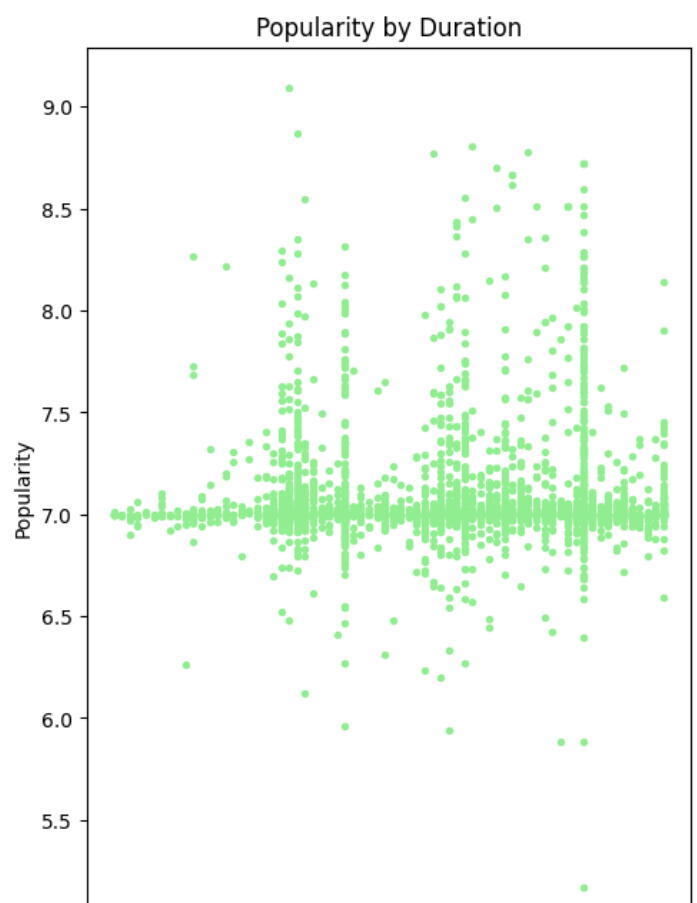
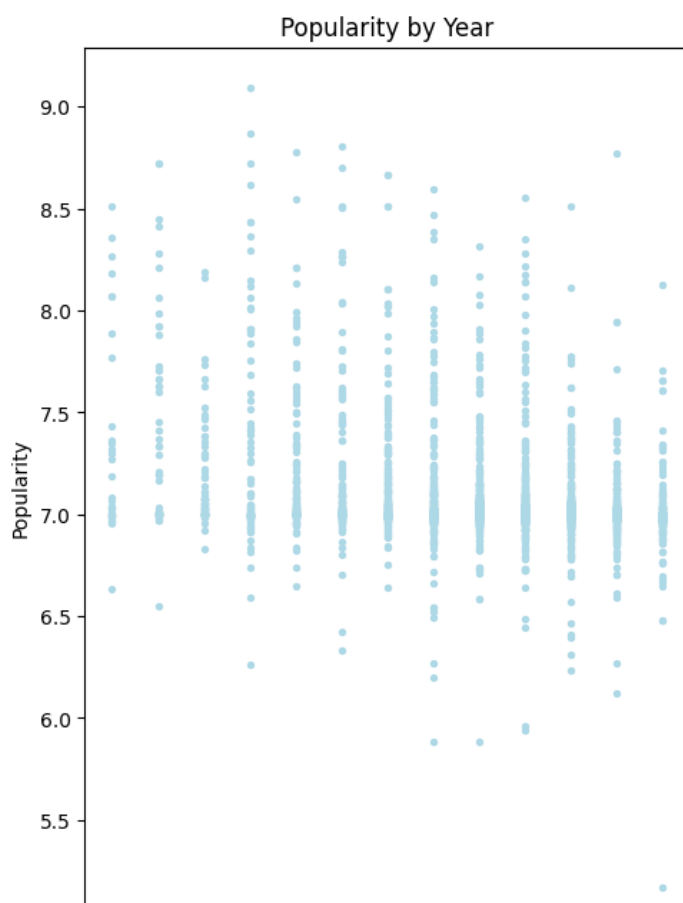
Visualizations -

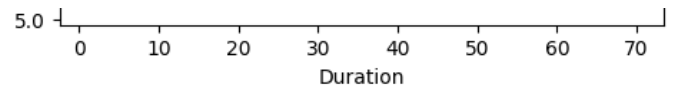
In [25]:

```
# Check for correlation between featurs -
```

```
fig, axs = plt.subplots(nrows = 1, ncols = 2, figsize = (12 , 8))
```

```
axs[0].scatter(data_numer['year'], data_numer['popularity'], s = 8, color = 'lightblue')
axs[0].set_xlabel('Year')
axs[0].set_ylabel('Popularity')
axs[0].set_title('Popularity by Year')
axs[1].scatter(data_numer['duration'], data_numer['popularity'], s = 8, color = 'lightgreen')
axs[1].set_xlabel('Duration')
axs[1].set_ylabel('Popularity')
axs[1].set_title('Popularity by Duration')
plt.show()
```





In [26]:

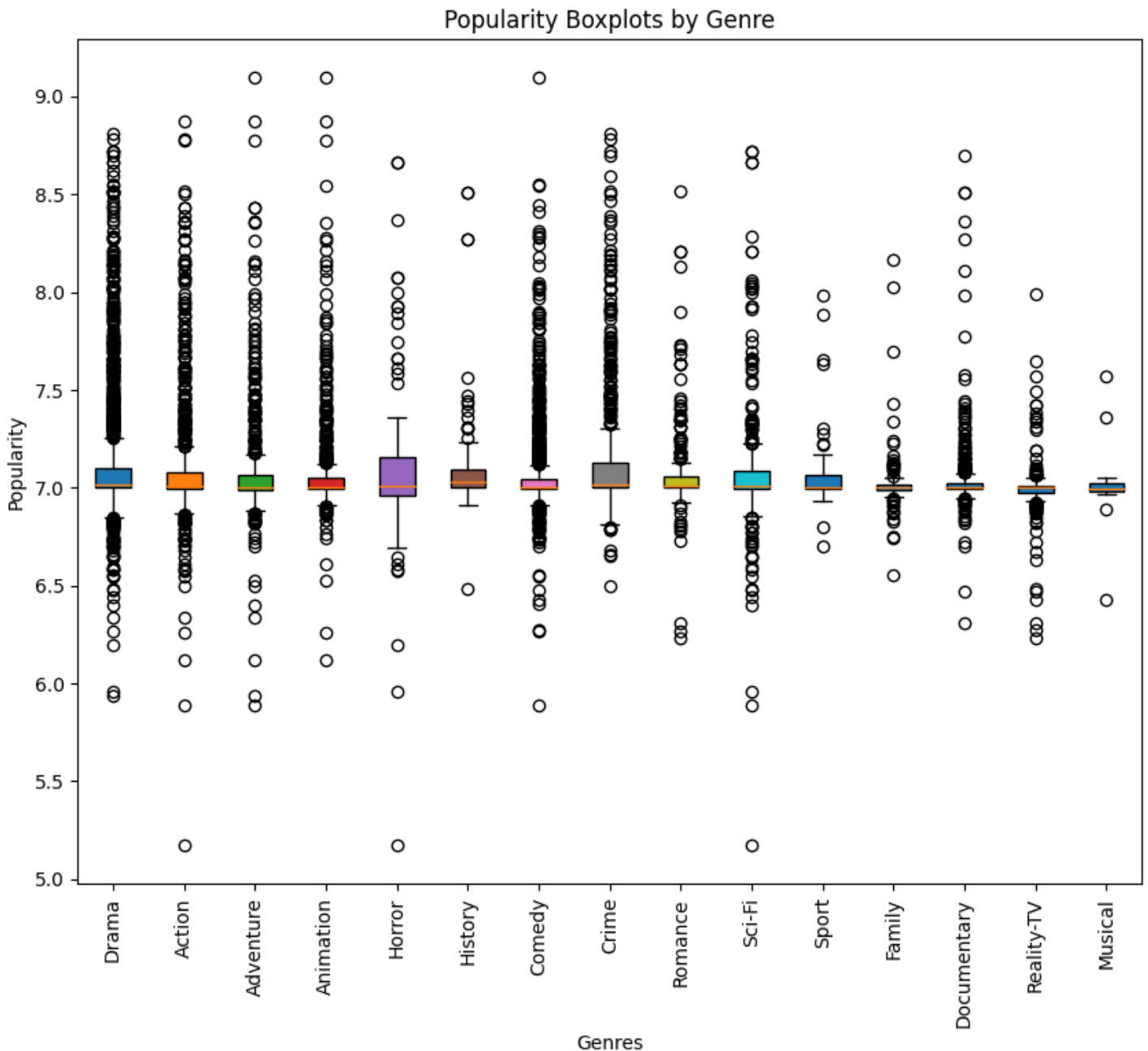
```
colors = plt.rcParams['axes.prop_cycle'].by_key()['color']
fig, ax = plt.subplots(figsize = (10, 8))

genres = ['Drama', 'Action', 'Adventure', 'Animation', 'Horror', 'History', 'Comedy', 'Crime', 'Romance', 'Sci-Fi', 'Sport', 'Family', 'Documentary', 'Reality-TV', 'Musical']
boxplot_data = [data_numer['popularity'][data_numer[c] == 1] for c in genres]

boxplot = ax.boxplot(boxplot_data, patch_artist = True)

for patch, color in zip(boxplot['boxes'], colors[: len(genres)]):
    patch.set_facecolor(color)

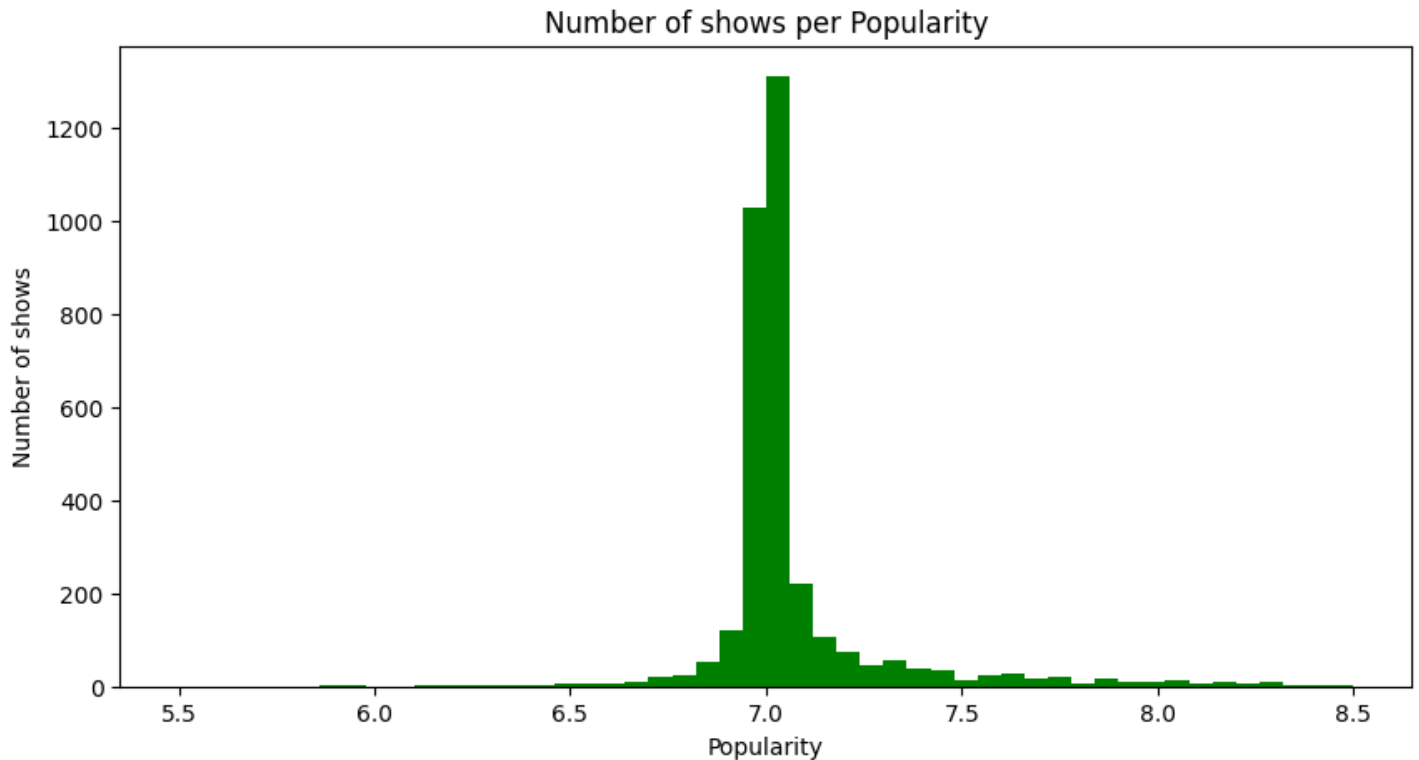
ax.set_xticks(range(1, len(genres) + 1))
ax.set_xticklabels(genres, rotation = 90)
ax.set_title('Popularity Boxplots by Genre')
ax.set_xlabel('Genres')
ax.set_ylabel('Popularity')
plt.show()
```



In [27]:

```
plt.figure(figsize = (10 , 5))
```

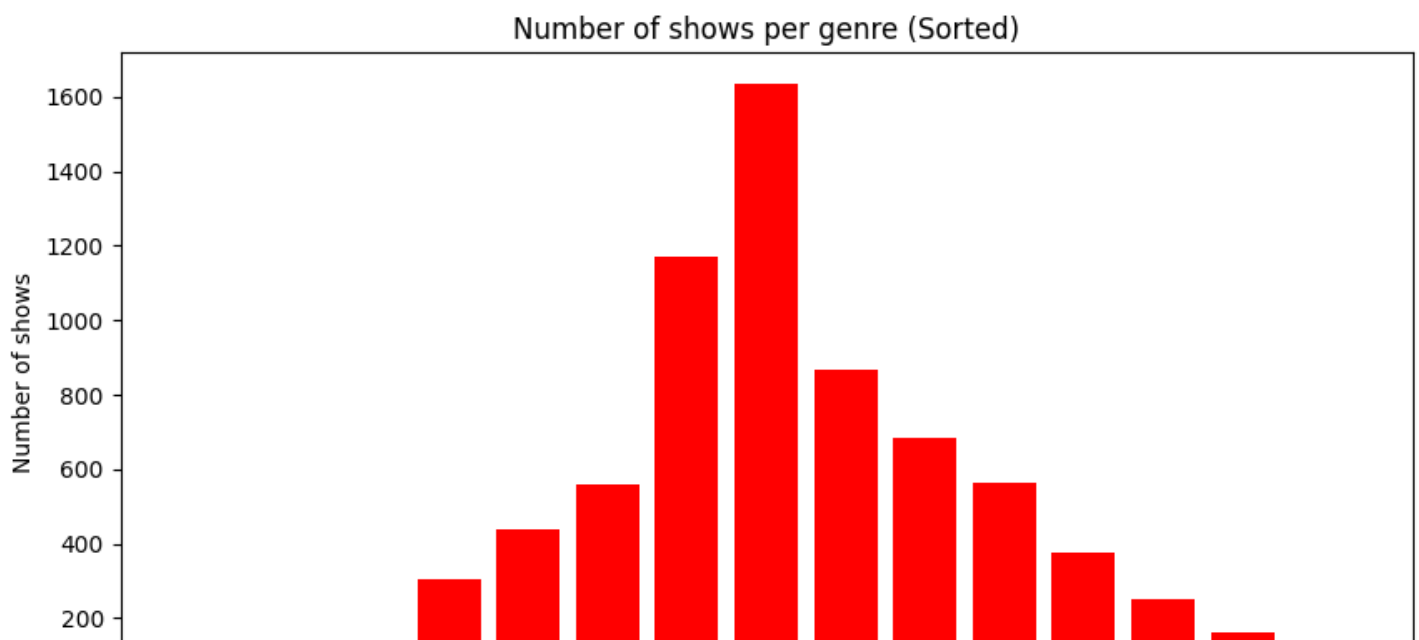
```
plt.xlabel('Popularity')
plt.ylabel('Number of shows')
plt.hist(data_num['popularity'], bins = 50, color = 'green', range = (5.5 , 8.5))
plt.title('Number of shows per Popularity')
plt.show()
```

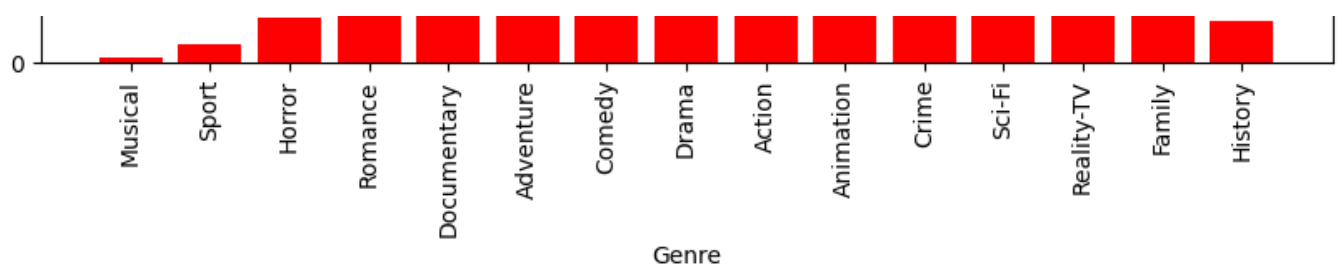


In [28]:

```
genres = ['Drama', 'Action', 'Adventure', 'Animation', 'Horror', 'History', 'Comedy', 'Crime', 'Romance', 'Sci-Fi', 'Sport', 'Family', 'Documentary', 'Reality-TV', 'Musical']
values = data_num[genres].sum()
tup_list = [(genres[i], values[i]) for i in range(len(genres))]
new = sorted(tup_list[:2], key = lambda x : x[1]) + sorted(tup_list[1:2], reverse = True, key = lambda x : x[1])
genres = [new[i][0] for i in range(len(new))]
values = [new[i][1] for i in range(len(new))]

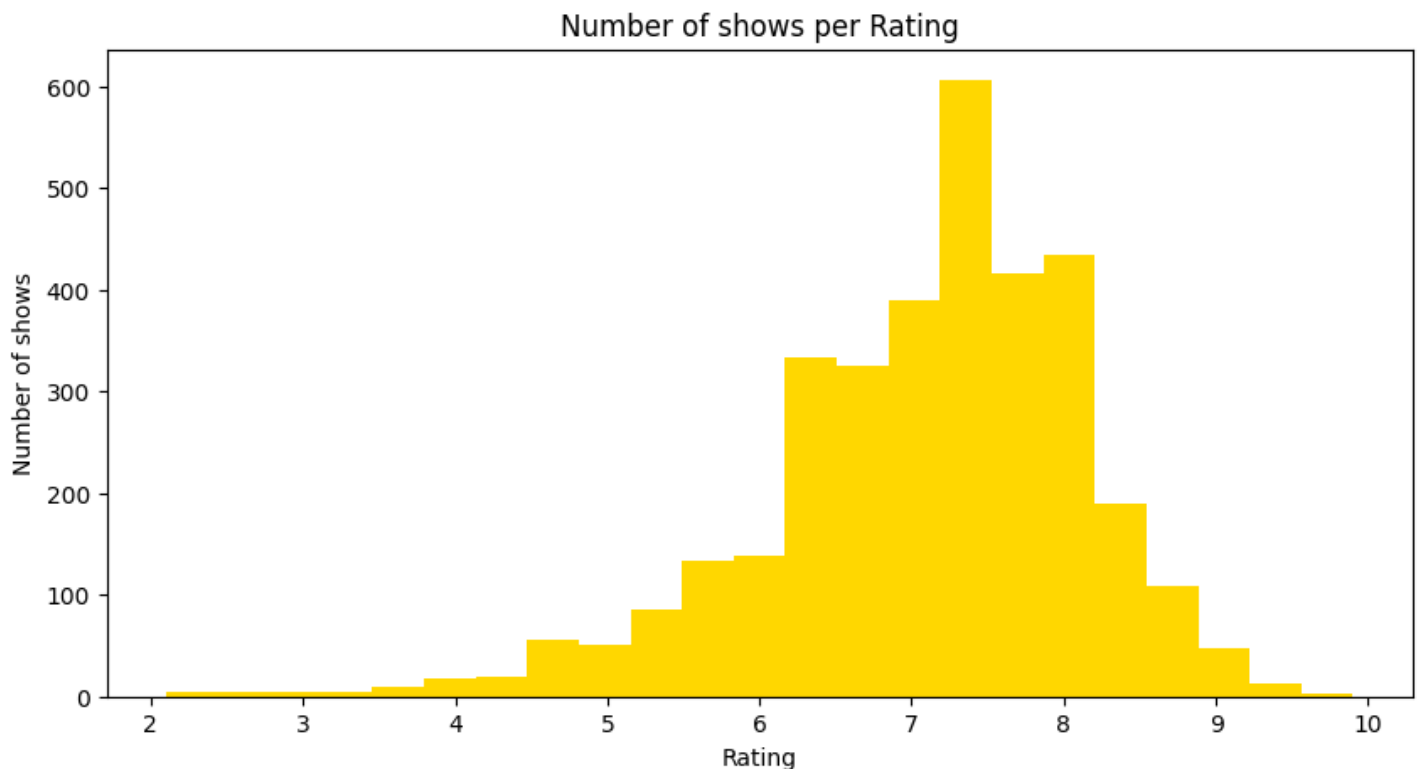
plt.figure(figsize = (10, 5))
plt.bar(genres, values, color = 'red', align = 'center')
plt.title('Number of shows per genre (Sorted)')
plt.xlabel('Genre')
plt.ylabel('Number of shows')
plt.xticks(genres, rotation = 90, fontsize = 10)
plt.show()
```





In [29]:

```
plt.figure(figsize = (10 , 5))
plt.xlabel('Rating')
plt.ylabel('Number of shows')
plt.hist(data_numer['rating'], bins = 23, color = 'gold')
plt.title('Number of shows per Rating')
plt.show()
```



In [30]:

```
categories = ['TV-PG', 'TV-TE', 'TV-MA', 'TV-NA']
certificate_counts = [10, 3, 5, 2]

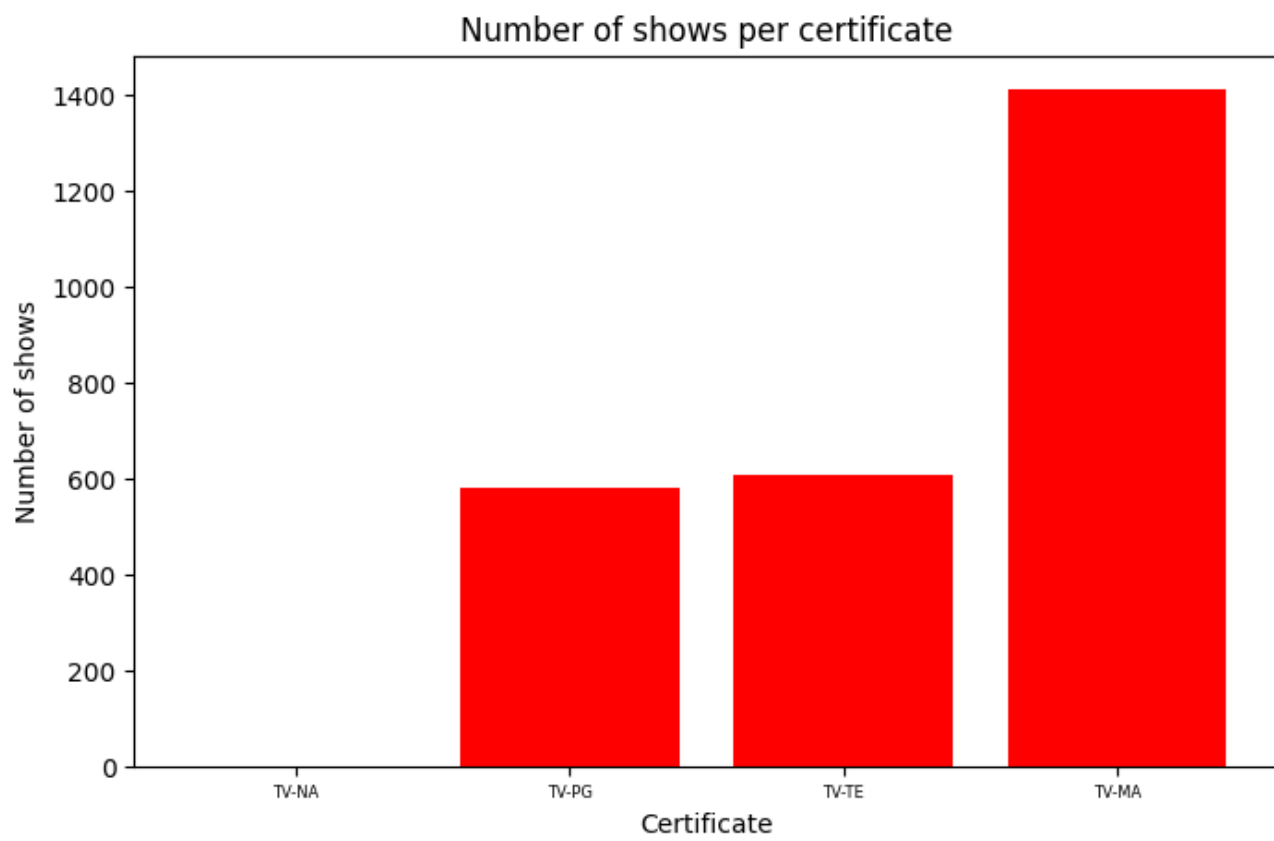
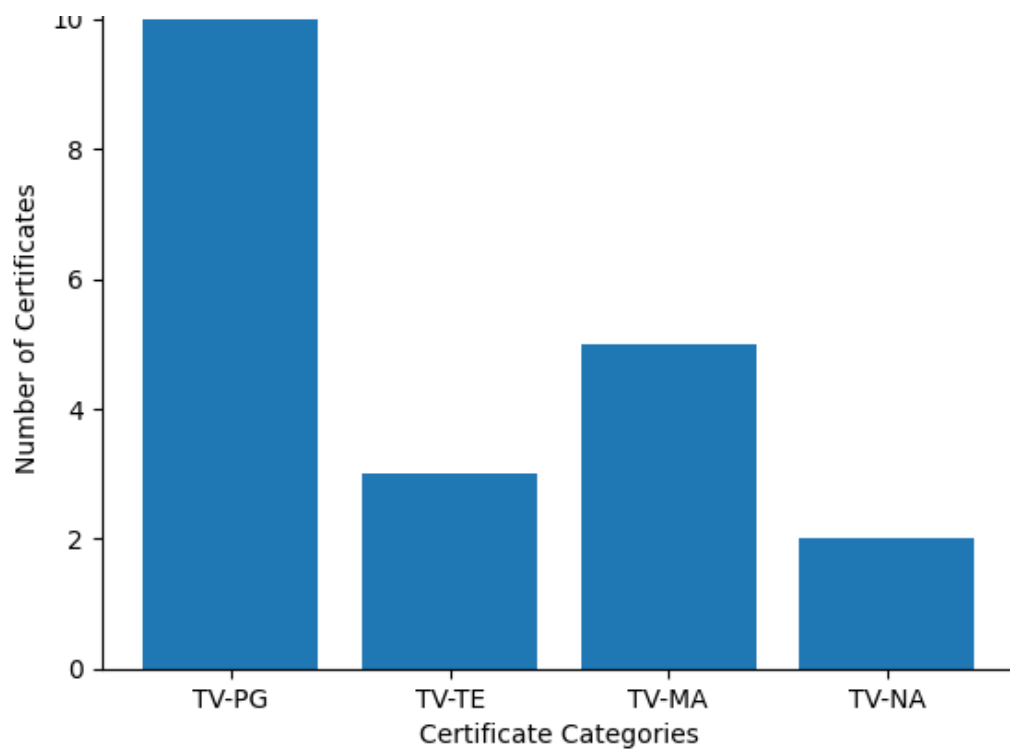
# Plotting the data
plt.bar(categories, certificate_counts)
plt.xlabel('Certificate Categories')
plt.ylabel('Number of Certificates')
plt.title('Consolidation of Certificates')

plt.show()

categories = ['TV-PG', 'TV-TE', 'TV-MA', 'TV-NA']

plt.figure(figsize = (8 , 5))
plt.bar(data_numer[categories].sum().sort_values().index, data_numer[categories].sum().sort_values().values, color = 'red')
plt.title('Number of shows per certificate')
plt.xlabel('Certificate')
plt.ylabel('Number of shows')
plt.xticks(fontsize=6)
plt.show()
```

Consolidation of Certificates



In []: